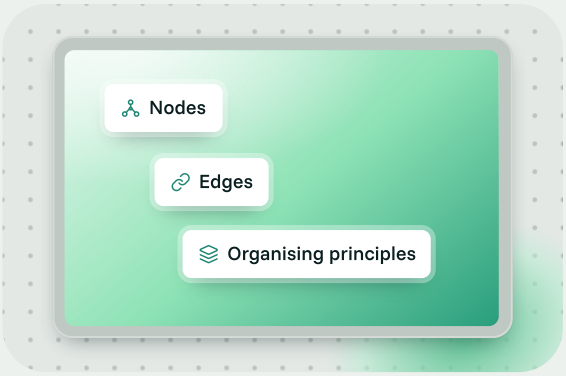
Last updated August 2026

Knowledge Graphs

How structured relationships between entities give AI systems grounded, queryable knowledge that goes beyond text retrieval.



14 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 6 of 10

What Is a Knowledge Graph?

In the previous guides, we covered how language models generate text, how prompts steer them, and how RAG retrieves documents to ground their answers. All of these approaches treat knowledge as text. A document is a bag of words, a chunk is a fragment of text, and retrieval is a search for text that looks similar to the query. This works well for many tasks, but it has a blind spot. Text captures what something says, but not how things relate.

Consider a simple question. Who is the CEO of the company that acquired the startup founded by the person who wrote this paper? A human can answer this by following a chain of relationships. They look up the paper, find the author, find the startup they founded, find the acquiring company, and find its CEO. Each step involves a specific relationship between two entities, and the answer comes from traversing that chain.

A knowledge graph stores exactly this kind of structured relationship. It is a data structure where entities are nodes and the connections between them are edges. Each edge carries a label that describes the relationship type. A node for a person connects to a node for a company through an edge labelled "founded." That company connects to another company through an edge labelled "acquired by." The CEO connects to the company through an edge labelled "leads." The graph captures not just what things are, but how they connect to each other.

The term knowledge graph was popularised by Google in 2012 as part of their search product, which used a graph of entities and their relationships to provide direct answers rather than just links to web

pages. Since then, the concept has spread far beyond search. Enterprises use knowledge graphs to organise internal data, researchers use them to map relationships in biology and drug discovery, and AI systems use them to ground language model outputs in verified facts.

Key principle

A knowledge graph separates knowledge from text. Text says things about the world. A knowledge graph encodes how the world is structured. This separation matters because structured relationships can be queried precisely, traced to their source, and updated without reprocessing unstructured documents.

Why Knowledge Graphs Exist

Relational databases store data in tables. A table of customers has columns for name, email, and address. A table of orders has columns for order ID, customer ID, and product ID. To answer a question like which products did this customer buy, you join the two tables on the customer ID column. This works, and it has powered business software for decades.

The problem appears when relationships become the focus of the question rather than a side effect of a join. Consider a fraud detection system. You want to know whether two accounts share a phone number, whether they transferred money to the same third account, whether that third account is linked to a known fraud ring, and whether any of these accounts share a device fingerprint. In a relational database, each of these connections is a join across a different table, and the query grows exponentially as you follow the chain. The database engine computes the joins at query time, which means deep relationship questions are slow and expensive.

A knowledge graph stores relationships directly. Rather than reconstructing connections through joins, the graph stores an explicit edge between each pair of related entities. Traversing the graph means following edges, which is a local operation. Finding all accounts connected to a given account within two hops is a graph traversal, and it is fast because the edges already exist in the data structure.

Relational Database	Knowledge Graph
Stores data in tables with rows and columns.	Stores data as nodes and edges in a network.
Relationships are reconstructed at query time using JOINs.	Relationships are stored explicitly as edges in the data.
Schema is rigid. Adding a new relationship type requires schema changes.	Schema is flexible. New relationship types can be added without restructuring existing data.
Deep multi-hop queries become slow as the number of joins grows.	Graph traversal handles multi-hop queries efficiently because edges are pre-stored.
Well suited for transactional workloads with fixed schemas.	Well suited for exploratory analysis, recommendation, and entity resolution.

None of this means knowledge graphs replace relational databases. For well-structured transactional data with a fixed schema, a relational database is usually the right choice. Knowledge graphs earn their place when relationships are the core of the question, when the schema evolves frequently, or when you need to integrate data from multiple sources with different structures.

How Knowledge Graphs Work

A knowledge graph has three structural components. Nodes represent entities. Edges represent relationships between entities. Properties store additional information about either a node or an edge.

Nodes

A node is a point in the graph that represents a discrete entity. A person, a company, a product, a document, a disease, or a gene can all be nodes. Each node has one or more labels that describe what kind of entity it is. A node might have the label Person, another might have the label Company, and a third might have the label Product.

Nodes can also carry properties. A Person node might have properties for name, date of birth, and nationality. A Company node might have properties for name, industry, and founding date. Properties are key-value pairs that store data attributes directly on the node.

Edges

An edge connects two nodes and describes the relationship between them. Every edge has a direction, meaning it goes from one node to another, and a type that describes the relationship. Common edge types include works at, founded, acquired, owns, located in, and reports to.

Edges can also carry properties. An edge of type acquired, connecting a company node to another company node, might have properties for the acquisition date and the deal value. This lets the graph store not just that a relationship exists, but the details of that relationship.

Organising Principles

A knowledge graph is more than a collection of nodes and edges. It needs an organising principle, meaning a conceptual framework that gives the graph structure and meaning. This could be a taxonomy that groups products into categories and subcategories, an ontology that defines what types of entities exist and what relationships are valid between them, or a set of business rules that govern how entities connect.

For example, an e-commerce knowledge graph might use a product taxonomy as its organising principle. Products are grouped into categories like Electronics, which contains subcategories like Phones and Laptops. Each product connects to its category through a belongs to edge. This structure lets the graph answer questions like what products are related to this one without scanning every product in the database.

Go deeper

Property graphs vs RDF triple stores

Optional

Two main data models exist for knowledge graphs. The property graph model, used by databases like Neo4j, stores nodes and edges that can both carry arbitrary properties. This makes it intuitive to model complex relationships because you can attach metadata directly to an edge. The RDF triple store model, rooted in the Semantic Web, stores everything as subject-predicate-object triples. Adding a property to a relationship requires creating additional triples, a process called reification, which can make the model verbose for highly connected data. Property graphs are more common in enterprise applications because they are easier to query and visualise, while triple stores remain relevant for interoperability with Semantic Web standards and linked open data.

Building a Knowledge Graph

Building a knowledge graph is a process that goes from defining the use case to ingesting data, querying it, and evolving the graph over time. The steps below describe the general pipeline.

Pascal Academy

Pascal Academy's knowledge graph workshop covers graph data modelling, entity resolution, and production deployment on graph databases. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Step	What Happens
1. Define use case	Choose a focused problem the graph will solve. Determine what entities and relationships are needed to answer the target questions.
2. Model the graph	Design the node types, edge types, and properties. Start with a rough model and plan to iterate after testing.
3. Gather and clean data	Collect data from multiple sources. Normalise formats. Resolve entities to eliminate duplicates and link the same entity across sources.
4. Ingest data	Load nodes and edges into the graph database. Apply the organising principle. Use schema mapping files to connect source data to the graph model.
5. Query and refine	Test with real queries. Identify missing relationships or data quality issues. Adjust the model and re-ingest as needed.
6. Evolve	Add new data sources, new entity types, and new relationship types over time. The graph grows as the use case expands.

Step 1. Define the Use Case

Start with a focused question. What problem will the knowledge graph solve? Common use cases include recommendation engines, fraud detection, supply chain tracking, entity resolution, and enterprise search. The use case determines what entities and relationships you need to model. A fraud detection graph needs accounts, transactions, devices, and phone numbers as nodes, with edges for transferred money to, shares device with, and shares phone number with. A recommendation graph needs users, products, and categories as nodes, with edges for purchased, viewed, and belongs to.

Choose a narrow starting point rather than modelling the entire domain. A focused scope lets you validate the approach with manageable effort. You can expand the graph later as the use case grows.

Step 2. Model the Graph

Graph modelling means deciding what nodes, edges, and properties your graph will contain. The goal is to represent the domain in a way that makes the important questions easy to answer. If the use case is recommendation, the model needs to support queries like which products did users similar to this one buy. That means the graph needs User nodes connected to Product nodes through purchased edges, and Product nodes connected to each other through shared category or co-purchase edges.

Unlike relational database design, graph modelling is iterative. You start with a rough model, load some data, query it, and refine the model based on what you find. The flexibility of the graph structure makes this iteration cheap. Adding a new relationship type does not require restructuring existing data.

Step 3. Gather and Clean Data

Knowledge graphs are only as good as the data they contain. Data comes from multiple sources, including relational databases, CSV files, APIs, unstructured documents, and third-party data providers. Each source needs to be cleaned and normalised before it can be loaded into the graph.

Entity resolution is the critical step at this stage. The same entity often appears in different sources with different names, formats, or identifiers. A company might be listed as IBM in one database, International Business Machines in another, and IBM Corp in a third. Entity resolution identifies these as the same entity and links them in the graph. Without this step, the graph contains duplicate nodes that fragment the relationship network.

Step 4. Ingest Data

Once data is cleaned and entities are resolved, the next step is to load it into the graph database. This involves creating nodes with their properties, then creating edges with their types and properties, and applying the organising principle. Graph databases typically provide bulk loading tools for large datasets and API-based loading for incremental updates.

Ingestion also involves mapping source data to the graph model. A schema mapping file defines how columns in a source table correspond to node properties and edge types in the graph. For example, a column called company_name in a source table maps to the name property on a Company node, and a column called parent_company_id maps to an edge of type subsidiary_of connecting two Company nodes.

Step 5. Query and Refine

After loading data, test the graph with real queries. Start with the questions defined in the use case. Do the queries return correct results? Are there missing relationships or duplicate nodes? Are the property values accurate? This testing phase often reveals data quality issues that were not visible during cleaning, and the graph model may need adjustment based on what the queries reveal.

Knowledge Graphs and AI

The connection between knowledge graphs and AI is where things get interesting for this series. Language models are good at generating text, but they have no inherent understanding of how entities relate to each other. They learn patterns from training data, and those patterns are statistical, not structural. A language model might know that Apple is a company and that Tim Cook is a person, but it has no reliable mechanism for knowing that Tim Cook is the CEO of Apple, that Apple acquired Beats Electronics, or that Beats was founded by Dr. Dre.

RAG addresses this by retrieving text documents that contain relevant information. But text retrieval has limitations. It finds documents that mention the query terms, not documents that are structurally related to the entities in the question. A retriever might find a document mentioning Apple and a document mentioning Beats Electronics, but it does not know that the relationship between them is an acquisition unless that exact fact appears in the retrieved text.

GraphRAG

GraphRAG combines knowledge graphs with retrieval-augmented generation. Instead of retrieving text chunks from a vector database, the system retrieves subgraphs from a knowledge graph. A subgraph is a set of nodes and edges that are relevant to the query. The retrieved subgraph is then converted to text and included in the prompt, giving the language model structured relationship data to reason over.

The advantage is precision. When a user asks about the supply chain for a specific product, a GraphRAG system traverses the graph from the product node, following edges to supplier nodes, then to sub-supplier nodes, and returns the full chain as a structured subgraph. The language model receives not just text about suppliers, but the actual relationships between products and suppliers, with edge properties like contract dates and order volumes.

Research from Neo4j and independent studies have shown that GraphRAG can reduce hallucination rates significantly compared to standard text-based RAG. One study reported that GraphRAG made AI

agents up to 80 percent more truthful on factual questions, because the structured relationships in the graph provide verifiable grounding that text alone cannot match.

Entity Linking

Entity linking is the bridge between unstructured text and a knowledge graph. Given a piece of text, entity linking identifies mentions of entities and connects them to the corresponding nodes in the graph. When a user asks about Tim Cook, the entity linking system finds the Person node for Tim Cook in the graph and returns its identifier. This identifier lets the system traverse the graph to find related entities, like the company Tim Cook leads and the products that company sells.

Entity linking is what makes GraphRAG practical. Without it, the system has no way to connect a user's natural language question to the structured data in the graph. Entity linking can be done using named entity recognition models, which identify entity mentions in text, followed by a disambiguation step that matches each mention to the correct node in the graph.



Go deeper

How GraphRAG retrieval works

Optional

In a standard RAG system, retrieval is a similarity search. The query is embedded into a vector, and the system finds text chunks whose vectors are closest to the query vector. In GraphRAG, retrieval is a graph traversal. The query is parsed to identify entities, those entities are matched to nodes in the graph, and the system traverses outward from those nodes following edges to find related entities. The resulting subgraph is then serialised into text, meaning the nodes and edges are converted into sentences like Company A acquired Company B in 2014 for 3 billion dollars. This text goes into the prompt alongside the user's question, and the language model generates its answer grounded in the structured relationships.

Knowledge Graphs vs Vector Databases

Vector databases and knowledge graphs are often discussed as alternatives, but they solve different problems. A vector database stores text chunks as vectors and finds similar chunks using distance metrics. It excels at semantic similarity, finding text that talks about the same topic even when the wording differs. A knowledge graph stores entities and their relationships and finds connected entities by traversing edges. It excels at structural queries, finding entities that are related through specific relationship types.

Pascal Academy

Pascal Academy's GraphRAG engineering course covers knowledge graph integration with RAG pipelines, entity linking, and hybrid retrieval architectures. Reach out to learn about team training options.

Ask about team training

Vector Database (Standard RAG)	Knowledge Graph (GraphRAG)
Stores text chunks as high-dimensional vectors.	Stores entities as nodes and relationships as edges.
Retrieval is similarity search. Finds text that is semantically close to the query.	Retrieval is graph traversal. Finds entities that are structurally connected.
Good for finding documents about a topic.	Good for finding entities related through specific relationships.
Cannot answer multi-hop questions without multiple retrieval passes.	Natively handles multi-hop questions by following edges.
No built-in notion of entity identity or relationship types.	Entity identity and relationship types are first-class citizens.
Easy to build. Chunk text, embed, store.	Requires more upfront modelling and entity resolution.

The two approaches are complementary. Many production systems use both. A vector database handles semantic retrieval, finding relevant text chunks for a given query. A knowledge graph handles structural retrieval, finding related entities and their connections. The results from both are combined in the prompt, giving the language model both textual context and structured relationship data to reason over.

Use Cases

Knowledge graphs have been deployed across industries. The examples below illustrate the range of problems they solve.

Fraud Detection

Financial institutions use knowledge graphs to detect fraud rings. Accounts, transactions, devices, phone numbers, and addresses are nodes. Edges connect accounts that share a device, transferred money to

each other, or used the same phone number. A fraud ring appears as a cluster of tightly connected nodes in the graph. Graph algorithms can detect these clusters automatically, flagging accounts that are part of a suspicious network even when no individual transaction looks fraudulent.

Supply Chain

Manufacturing companies use knowledge graphs to map their supply chains. Products, components, suppliers, and facilities are nodes. Edges represent supplies, manufactures, and ships to relationships. When a supplier experiences a disruption, the graph shows which products are affected and which alternative suppliers can fill the gap. This kind of analysis is difficult in a relational database because it requires traversing multiple levels of the supply chain, each involving joins across different tables.

Recommendation Engines

E-commerce platforms use knowledge graphs to power product recommendations. Users, products, and categories are nodes. Edges represent purchased, viewed, rated, and belongs to relationships. To recommend products to a user, the graph finds products that similar users purchased, products in the same category as previously purchased items, and products frequently co-purchased with items in the user's history. The graph structure makes these multi-hop queries efficient.

Enterprise Search and GraphRAG

Organisations use knowledge graphs to make internal documents searchable and to ground AI assistants in company data. Documents, people, projects, and departments are nodes. Edges connect documents to their authors, projects to their team members, and departments to their projects. When an employee asks an AI assistant about a project, the system traverses the graph to find related documents, team members, and dependencies, giving the language model structured context that text retrieval alone cannot provide.

Challenges and Limitations

Knowledge graphs are powerful, but they come with costs and limitations that are worth understanding before investing in one.

Data Quality and Maintenance

A knowledge graph is only as accurate as the data it contains. Entities change over time. People change jobs, companies merge, products are discontinued. If the graph is not updated, it becomes stale and its answers degrade. Maintaining a knowledge graph requires ongoing data quality processes, including entity resolution for new data, deduplication, and validation of relationships. This is a continuous operational cost, not a one-time setup task.

Building Effort

Building a knowledge graph requires more upfront effort than setting up a vector database. You need to design a graph data model, choose an organising principle, build entity resolution pipelines, and implement ingestion logic. For a team that just needs to answer questions from a set of documents, standard RAG with a vector database is faster to build and simpler to maintain. A knowledge graph earns its investment when the use case involves complex relationships, multi-hop queries, or the need to integrate structured data from multiple sources.

Query Complexity

Querying a knowledge graph requires a different skill set than querying a vector database. Graph query languages like Cypher and SPARQL have their own syntax and semantics. Writing efficient graph queries requires understanding traversal patterns, index usage, and query optimisation. Teams without graph database experience will need training or external expertise to get productive.

Scalability

Graph databases scale differently from relational databases. As the graph grows, query performance depends on how well the traversal patterns align with the graph structure and indexes. Some queries that perform well on a small graph become slow on a large one because the traversal touches too many nodes. Sharding a graph across multiple servers is harder than sharding a relational table because relationships can cross server boundaries. Distributed graph databases exist, but they add complexity and have trade-offs in query performance.

When to Use a Knowledge Graph

The decision to build a knowledge graph depends on the nature of the questions you need to answer and the structure of your data. The table below maps common situations to recommendations.

Situation	Recommendation
Answering questions from a set of documents	Use standard RAG with a vector database. Faster to build and sufficient for text-based questions.
Answering questions that require multi-hop reasoning across entities	Use a knowledge graph or GraphRAG. The graph stores relationships explicitly, making multi-hop queries efficient and accurate.
Detecting fraud rings or suspicious networks	Use a knowledge graph. Graph algorithms excel at finding clusters of connected entities that indicate coordinated activity.
Product recommendations based on user behaviour	Use a knowledge graph. User-product interactions, category hierarchies, and co-purchase patterns are naturally graph structures.
Integrating structured data from multiple sources with different schemas	Use a knowledge graph. Entity resolution links the same entity across sources, and the flexible schema accommodates new data types without restructuring.
Simple keyword search over a document corpus	Use a vector database or traditional search index. A knowledge graph is overkill when the question is just finding documents about a topic.

The Future of Knowledge Graphs in AI

Knowledge graphs are becoming increasingly relevant as AI systems move beyond simple text generation toward reasoning and decision-making. Language models can generate plausible text, but they lack the structured knowledge needed to verify facts, trace relationships, and avoid hallucination. Knowledge graphs provide that structure.

The integration of knowledge graphs with LLMs is still early. Many production systems use graphs as a retrieval source, converting subgraphs to text and including them in prompts. Research is active on

tighter integration, where language models can directly query a graph during generation, traversing edges and reading properties as part of their reasoning process. This would give models real-time access to structured knowledge without the overhead of converting everything to text.

Another direction is automated graph construction. Building a knowledge graph currently requires significant manual effort for entity resolution, relationship extraction, and data cleaning. Language models can assist with these tasks by extracting entities and relationships from unstructured text, disambiguating entity mentions, and suggesting relationship types. This could lower the cost of building and maintaining knowledge graphs, making them accessible to a wider range of applications.

Summary

A knowledge graph is a data structure that stores entities as nodes and their relationships as edges. Unlike relational databases, which reconstruct relationships through joins at query time, knowledge graphs store relationships explicitly, making multi-hop queries efficient. Building a knowledge graph involves defining a use case, modelling the graph, gathering and cleaning data, resolving entities across sources, and ingesting data into a graph database. In AI applications, knowledge graphs complement text-based RAG by providing structured relationship data through GraphRAG, where subgraphs are retrieved and included in the prompt alongside text. Knowledge graphs excel at fraud detection, recommendation, supply chain analysis, and enterprise search, where the core question involves relationships between entities rather than just text similarity. They require more upfront effort than vector databases but provide precision and verifiability that text retrieval cannot match.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, context windows, and knowledge graphs. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is a knowledge graph in AI?

A knowledge graph is a structured representation of information where entities are nodes and relationships are edges. It stores facts as connections between things, e.g. Company A competes with Company B, or Executive C works at Company D. This structure makes it possible to query relationships that would be hard to extract from text alone.

How are knowledge graphs used in finance?

Knowledge graphs help finance teams map relationships between companies, executives, suppliers, customers, and sectors. They support competitive analysis, supply chain mapping, risk detection, and finding hidden connections that might affect an investment thesis.

What is GraphRAG and how does Pascal use knowledge graphs?

GraphRAG combines knowledge graphs with retrieval-augmented generation. Instead of retrieving text chunks, the system retrieves connected entities and relationships from a graph, then uses a language model to synthesize the results. Pascal builds a persistent knowledge graph across every document and filing your team touches, so research compounds over time and every query gets smarter.

What is the difference between a knowledge graph and a vector database?

A vector database stores text chunks as embeddings and retrieves semantically similar passages. A knowledge graph stores structured relationships between entities. Vector databases are good for finding relevant text. Knowledge graphs are good for answering questions about how things connect. They serve different purposes and can complement each other.